CORE JAVA SERIES — DAY 9

# Abstraction, Polymorphism & Class Internals

The last two pillars — abstraction (low-level and high-level, via abstract classes and interfaces) and polymorphism (compile-time overloading, run-time overriding); plus class internals: why one public class per file, wrapper classes, autoboxing/unboxing, the Integer cache, `==` vs `equals`, abstract-class interview questions, and POJO/domain models.

## Abstraction — The Third Pillar

**Definition:** *the process of focusing on **what** something does, while ignoring **how** it does it.* The entire human population runs on abstraction: nobody can process full reality, so we model only what is **necessary** and hide everything else.

**The car example.** A driver knows a car *accelerates, brakes, starts* — the *what* — but not the ~30,000 parts or the combustion engine behind it — the *how*. If abstraction didn't exist, every human would have to be an engineer, every engineer a physicist, every physicist a mathematician. The ATM machine is another case: users know *deposit, withdraw, check balance*; nobody needs its electronics.

> **A half-truth corrected:** OOP does not simulate reality *as it is* — it simulates **what humans think about reality**. We model a Teacher by a `teach()` behavior, not by modeling every cell and organ. **Model whatever is necessary; hide the rest.** Even when we model something (a car's accelerate/brake/clutch), we don't want the whole world to know *how* it works — we want them to be able to **use** it. The manufacturer hides the internals behind an **interface** — steering wheel, pedals, gear stick.

### Two levels of abstraction in Java

| Level | Idea | How Java implements it |
|---|---|---|
| **Low-level** | Hide implementation details from the user of a class. | Ordinary classes — methods with bodies; callers write `c.accelerate()` without knowing the body. (This is exactly encapsulation's bundling of data + behavior.) |
| **High-level** | **Separate the WHAT from the HOW** — decouple the contract from the implementation. | **Abstract classes** and **interfaces** |

## High-Level Abstraction via Inheritance — The Problem

In a concrete `Car` class, `Car c = new Car();` is **tightly coupled**: `c.accelerate()` always runs *this* accelerate. The what and the how are welded together. But a FuelCar and an ElectricCar accelerate **differently** — so we get a hierarchy:

```
class Car {                          // parent
    void applyHandBrake() { ... }  // common — implemented once here
    void accelerate() { ... }      // generic... but WHOSE logic?
    void brake() { ... }
}
class FuelCar extends Car { ... }   // overrides accelerate(), brake()
class ElectricCar extends Car { ... }
```

A generic `Car` has no meaningful accelerate/brake implementation — every concrete car type implements them differently. Yet with concrete classes, Java **forces** us to write a dummy body for them, which every child then silently inherits unless overridden. This is exactly what abstract classes fix.

## Abstract Classes

```
abstract class Car {
    void start() { ... }                // normal (concrete) method — common to all
    abstract void accelerate();         // declared, NOT defined
    abstract void brake();
}
class FuelCar extends Car {
    @Override void accelerate() { ... }  // forced to define
    @Override void brake() { ... }
}
```

**Rules.** (1) A method declared `abstract` has **no body**. (2) If a class has *even one* abstract method, the class itself **must** be declared `abstract`. (3) An abstract class **cannot be instantiated** — `new Car()` fails with "*cannot instantiate the type Car*" (calling an undefined accelerate would be a contradiction). (4) Therefore it is **meant to be extended**: a subclass must give definitions to all inherited abstract methods, or be declared abstract itself. (5) Mixed content is allowed — concrete methods (like `start()`) alongside abstract ones.

```
Car c = new ElectricCar();  // parent reference → child object ✔
c.start();                  // parent's common method
c.accelerate();             // ElectricCar's version
c.brake();                  // ElectricCar's version
```

The **what** (accelerate, brake, start) is now separated from the **how** (defined per concrete car type). The `@Override` annotation is optional but good practice — it tells the compiler the method intentionally overrides the parent's.

## Interfaces

```
interface Car {                 // keyword: interface, not class
    void start();               // public abstract by default — no keyword needed,
    void accelerate();          // no body allowed (pre-Java-8 view)
    void brake();
}
class FuelCar implements Car { ... }    // implements, not extends
class ElectricCar implements Car { ... }
```

- An interface is **not a blueprint of an object** — it is a **contract**. It cannot be instantiated (`new Car()` → compile error).
- Classes **extend** classes; classes **implement** interfaces.
- All interface methods are `public abstract` by default, and in classic Java **none** may have a body (default/static methods came later — deferred).
- **Visibility trap:** overriding methods must stay `public` — you *cannot reduce visibility* of an inherited method ("*cannot reduce the visibility of the inherited method*").

### Interface vs Abstract class — the intention

| | Abstract class | Interface |
|---|---|---|
| **Represents** | A **family of similar objects** (FuelCar, ElectricCar *are* Cars) | **Roles / responsibilities / capabilities** — a contract of what an implementer must provide |
| **Content** | Partial what + how — concrete and abstract methods mix | Pure what (classic view) — declarations only |
| **Keyword** | `extends` | `implements` |
| **Multiple inheritance** | Not supported | **Supported** — the workaround for Java's multiple-inheritance ban (details with interfaces deep-dive) |

> **Naming conventions.** Interfaces are usually named for the capability they confer — `Flyable`, `Walkable`, `Runnable`, `Comparable`, `Serializable` (note the JDK's `-able` pattern). `Car` is a poor interface name (it is a family); `Drivable` would fit better. C# prefixes interfaces with `I` (`ICar`) — not a Java rule, but some teams follow it.
>
> **Design decision:** if the parent mostly groups a **family** of similar objects → abstract class; if it mostly states **rules a class must follow** → interface. Subjective, but the intention differs.

## Active Recall — Part 1

**Q1. Define abstraction in one line.**

A. Focusing on *what* something does while ignoring *how* it does it.

**Q2. What forces a class to be declared abstract?**

A. Having even one abstract method (no body) in it.

**Q3. Why can't an abstract class be instantiated?**

A. Its abstract methods have no bodies — an object whose method could be called with no implementation would be a contradiction.

**Q4. Interface vs abstract class in terms of intention?**

A. Abstract class = family of similar objects (may share some implementation); interface = a contract of roles/responsibilities/capabilities.

## Encapsulation vs Abstraction

| Encapsulation | Abstraction |
|---|---|
| Bundles data + methods together in a class **and** talks about **data security** via access modifiers (private fields, controlled access). | Talks about **hiding implementation details / data** so users only see what something does, not how. |
| Provides **security**. | Provides **simplicity** — no security claim. |

You can open a car's bonnet and study its engine — nothing is "secured" from you; it's simply *hidden by default* because you don't need it. Encapsulation secures; abstraction hides.

## Polymorphism — The Fourth Pillar

*Poly* = many, *morphism* = forms → "many forms." Two real-life patterns, two programmatic mechanisms:

### 1. Same object, same command, different behavior (compile-time)

A human told "*run*" jogs at 2 km/h; the same human told "*run*" with a dog behind him sprints at 5 km/h. Same object, same stimulus — one parameter changed, response differs.

```
class Human {
    void run()            { ... }   // 2 km/h
    void run(boolean dogBehind) { ... }  // 5 km/h if dogBehind
}
```

**Method overloading** — the compiler decides *at compile time* which `run()` runs, purely from the call site: `run()` → first, `run(true)` → second. Hence **compile-time polymorphism** (static binding).

### 2. Different objects, same command, different behavior (run-time)

Tell a Dog, a Duck, and a Human — all Animals — to *run*: each runs differently, even in identical situations. Programmatically:

```
abstract class Animal { abstract void run(); }
class Dog   extends Animal { void run() { ... } }
class Duck  extends Animal { void run() { ... } }
class Human extends Animal { void run() { ... } }

Animal a = new Dog();   // or new Duck() / new Human()
a.run();                // which run()? decided at RUN TIME
```

**Method overriding** — the object is created dynamically (`new` at run time, possibly selected by a condition like `if (type.equals("duck")) a = new Duck();`), so the JVM resolves the call *while the program runs*. Hence **run-time polymorphism** (dynamic binding).

| Type | Mechanism | Decided when |
|---|---|---|
| Compile-time (static) | Method **overloading** — same name, different parameters | Compile time, from the call site |
| Run-time (dynamic) | Method **overriding** — child redefines parent's method | Run time, from the actual object's class |

Polymorphism is "nothing new" — it is exactly overloading and overriding, which we already used throughout the OOP lectures.

## Active Recall — Part 2

**Q1. Which polymorphism type does overloading belong to, and why?**

A. Compile-time — the compiler picks the method from the arguments at the call site.

**Q2. Why is overriding resolved at run time?**

A. The actual object (`new Dog()` vs `new Duck()`) only exists at run time, so the JVM picks the method then.

**Q3. Encapsulation vs abstraction in one sentence each?**

A. Encapsulation = bundle data+methods and secure data with access modifiers; abstraction = hide implementation so only "what" is visible.

## What Can and Cannot Be Overridden

| Member | Overridable? | Why |
|---|---|---|
| `static` method | **No** | Static members **belong to the class, not the object**. Binding is to the *reference's type*: `A a = new B(); a.fun();` always calls **A's** static fun, never B's — the object being pointed to is irrelevant. |
| `private` method | **No** | Invisible outside the class — the child cannot even see it, so it cannot override it. A same-named method in the child is simply a **different, unrelated method**. |
| `final` method | **No** | That is the keyword's purpose: "*cannot override the final method from A.*" Use it when a parent method must be used as-is by every child. |
| `final` class | **Cannot be extended at all** | `final class A` → "*The type B cannot subclass the final class A.*" Use it when a class must never be inherited. |
| Fields (variables) | **Not polymorphic** | Fields don't participate in run-time dispatch — access is bound to the **reference type**, not the object. |

```
A a = new B();        // A has int x = 10;  B has int x = 20;
System.out.println(a.x);   // prints 10 — A's field, because a is an A reference
```

If `x` were returned by an overridden `getX()` method instead, `a.getX()` would print 20 — **methods dispatch by object, fields resolve by reference type.**

## Why Only One public Class Per File?

> **Rule.** A `.java` file may contain at most one `public` class, and its name must equal the file name (`Demo.java` → `public class Demo`). Any number of non-public classes may coexist. Violations: "*The public type Sample must be defined in its own file.*"

**The reasoning (trace the JVM's job):**

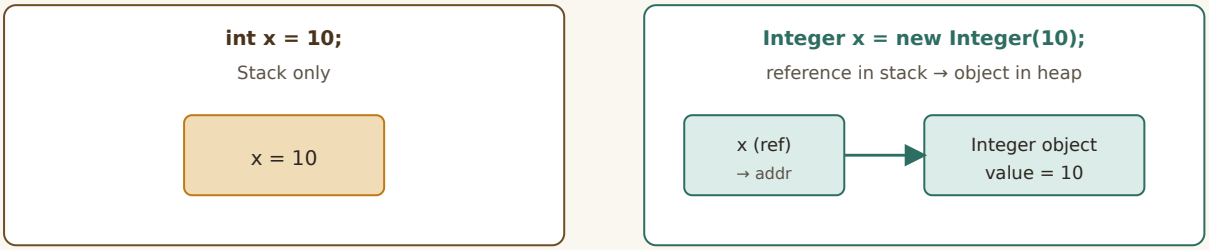
- The JVM must locate and call `main` to start the program. `main` is `public` so the JVM (outside the package) can see it, and `static` so no object is needed.
- If several public classes existed, the JVM could not know *which* one holds `main` — `Demo.main()`? `Sample.main()`? With exactly one public class, there is no ambiguity: the only class visible outside the package is the entry holder.
- Matching the class name to the file name lets the JVM load it trivially: see file `Demo.java` → load `Demo` → call `Demo.main()`.
- In a real application with hundreds of files, the file you **run** is the one containing `main` (imports pull the rest in) — so the JVM never has to search.

## Wrapper Classes

Java's data types split into primitives and non-primitives. For **every primitive, Java supplies a wrapper class** — a non-primitive substitute that stores the value *plus* provides methods:

| Primitive | Wrapper | Primitive | Wrapper |
|---|---|---|---|
| `int` | `Integer` | `double` | `Double` |
| `long` | `Long` | `char` | `Character` |
| `short` | `Short` | `boolean` | `Boolean` |
| `float` | `Float` | `byte` | `Byte` |

- **Why wrappers exist:** the **Collection Framework** (ArrayList, LinkedList, …) works only with objects/classes, never with primitives; and an object-oriented language wants a substitutable object form for every value, with extra methods on top.
- **Why primitives still exist:** (1) **legacy** — C/C++ compatibility attracted existing developers when Java was designed; (2) **speed & memory** — `int x = 10` is one stack container, while an object costs heap block + header + padding + a stack reference. Primitives stay the default; wrappers appear mainly with collections.




Primitive: a single stack container. Wrapper: reference variable + heap object (header, padding) — heavier, but an object with methods.

## Autoboxing & Unboxing

**Autoboxing** — a primitive converts *automatically* to its wrapper, no cast or syntax needed:

```
int x = 10;
Integer y = x;        // autoboxing — compiler rewrites it internally as:
                      // old Java:  Integer y = new Integer(x);
                      // modern :  Integer y = Integer.valueOf(x);
```

**Unboxing** — the reverse, wrapper → primitive:

```
Integer x = 10;
int y = x;            // unboxing — compiler rewrites as: int y = x.intValue();
```

### Where it applies automatically

| Context | Example |
|---|---|
| Assignment statements | `Integer y = x;` / `int y = x;` |
| Method calls (arguments vs parameters) | `print(50)` into `void print(Integer i)` (boxing) or `print(x)` with `Integer x` into `void print(int i)` (unboxing) |
| Arithmetic operations | `int sum = a + b;` with `Integer a, b` — both unbox, add, box into `sum` if declared `Integer` |
| Printing an object | `System.out.println(y)` — `y` (an object) silently unboxes via `intValue()` |

**Null danger.** `Integer x = null;` is legal (references hold null), but `int y = x;` compiles fine and **throws NullPointerException at run time** — internally `x.intValue()` is called on null, and primitives have no notion of null. Never unbox a potentially-null wrapper.

## Active Recall — Part 3

**Q1. Why do wrapper classes exist, and why do primitives still exist?**

A. Collections and much of the Java API work only with objects → wrappers; primitives remain for legacy compatibility and for speed/memory efficiency.

**Q2. What does the compiler rewrite `Integer y = x;` (int x) into?**

A. `Integer y = Integer.valueOf(x);` (modern Java; old Java used `new Integer(x)`, now deprecated).

**Q3. Why does `int y = x;` (Integer x = null) throw at run time?**

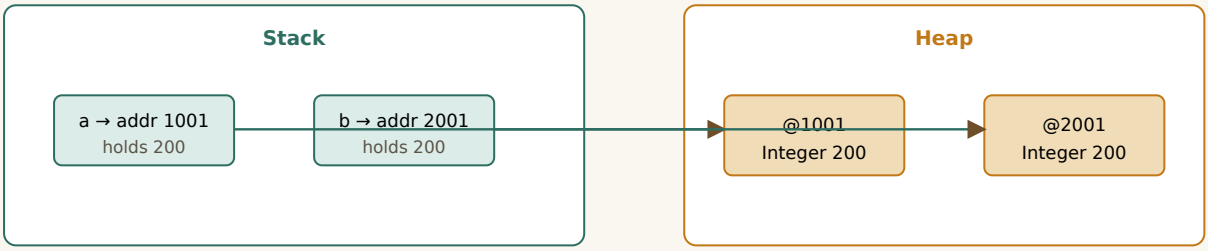
A. It becomes `x.intValue()` — calling a method on null → NullPointerException.

## == vs equals on Wrappers, and the Integer Cache

```
int x = 100, y = 100;
System.out.println(x == y);          // true — values compared

Integer a = 200, b = 200;
System.out.println(a == b);          // FALSE — references compared!
System.out.println(a.intValue() == b.intValue());  // true
System.out.println(a.equals(b));                   // true
```

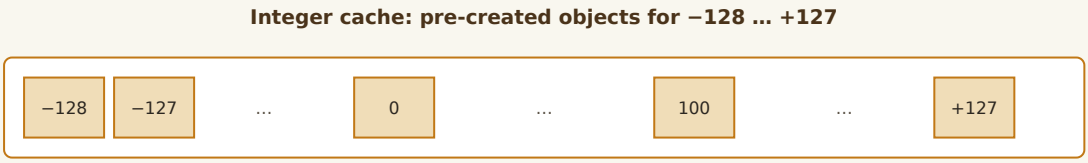
**First-principles view:** `x == y` on primitives compares the values stored *in* the containers — there are no objects. `a == b` on objects asks "do these two **reference variables** hold the same address?" — each `Integer.valueOf(200)` produced a distinct heap object, so the addresses differ. Nothing about `==` changed; *you* asked about references, not values. To compare contents, either unwrap ( `intValue()` ) or call `a.equals(b)` .




a and b point to two distinct Integer objects (same content, different addresses) → a == b is false.

### The cache twist: -128 to 127

`Integer a = 100, b = 100; a == b` → **true** (compiler- and JVM-dependent in theory, true in practice). Reason: `valueOf()` **caches** Integer objects for the entire `byte` range **-128 … +127** — the most commonly used values. Those objects are pre-created at class-load time; `valueOf(100)` doesn't allocate, it just returns the *existing* reference ( `a = h; b = h;` internally). Both references then point to the **same object** → `==` true. Outside this range, new objects are allocated each time.




Integer x = 100 → x = valueOf(100) → returns the pre-made object's reference (no new allocation)

Integer x = 200 → outside cache → a brand-new object is allocated every time

Caching is exactly why valueOf() is preferred over the deprecated new Integer(...).

### Conceptual Integer class (reconstructed)

```
public final class Integer {              // final: nobody may subclass the built-in type
    private int value;                    // the primitive inside, hidden
    public Integer(int value) { ... }     // deprecated since Java 9 — use valueOf
    public int intValue()  { return value; }                 // unboxing helper
    public static Integer valueOf(int x) { ... }              // caching lives here
    public boolean equals(Integer y) { return this.value == y.value; }
}
```

The real JDK class is far richer (it extends a parent class — where `equals` actually comes from — implements interfaces, holds arrays), but this captures everything its behavior implies. After finishing the

series you can open any JDK class and read it line by line.

## Abstract Classes — Interview Deep Dive

| Question | Answer | Reasoning |
|---|---|---|
| Constructors allowed? | **Yes** | When a child object is created, the parent's constructor runs (via `super(...)`) to initialize inherited fields — e.g. `Animal(String name)` called from `Dog(String name) { super(name); }`. |
| Can it be final? | **No** | `final` = "cannot be inherited"; abstract = "must be inherited to be useful". Contradiction — compiler: "*class can be either abstract or final*." |
| Static methods/variables/blocks? | **Yes** | Statics belong to the *class*, which exists even when no object can be created. (But a static method can't be abstract — again nothing could ever define it.) |
| Private methods? | **Yes — but non-abstract only** | A private helper is fine; an abstract private method is impossible (child can't see it to define it). |
| Final methods? | **Yes — but non-abstract only** | A final concrete method forces all children to use this implementation; final + abstract contradicts. |
| Abstract methods required? | **No** | An abstract class with zero abstract methods is legal — used to **block direct instantiation** while providing default implementations. |

## Active Recall — Part 4

**Q1. Why is `a == b` false for `Integer a = 200, b = 200`?**

A. `==` on references compares addresses; two separate objects were allocated. Use `equals()` or compare `intValue()`s.

**Q2. Why is the same comparison true for 100?**

A. valueOf() caches Integer objects for −128…+127, so both references get the same pre-made object.

**Q3. Can an abstract class have a constructor? Why does that make sense?**

A. Yes — child constructors chain to it via super(), so inherited fields get initialized.

**Q4. Can an abstract class be final, or have abstract private/final methods?**

A. No to all — each pairing is a contradiction (must-be-inherited vs cannot-be-inherited; must-override vs cannot-override).

## POJO & Domain Models

**POJO = Plain Old Java Object** — a simple class with **no external framework** (no Spring annotations, etc.): some fields, constructors, getters/setters; maybe a little business logic. POJOs are also called **model classes** — they model a real-world object (Student → name, age, rollNumber, college) so it can be carried around the program, stored in a DB table, sent over a network, etc. When you need to persist a Student in SQL, you need exactly such a class mapping columns to fields.

| Style | Contains |
|---|---|
| **Anemic model** | Only fields + constructors + getters/setters. No behavior. |
| **Rich domain model** | The above **plus business logic** (e.g. `markAttendance()` living inside Student). |

"POJO" is a **subjective** term — both styles above are still POJOs (no framework involved); the anemic/rich split is purely an architectural preference about where behavior lives.

*Core Java Series — Day 9 Notes · Abstraction · Abstract classes & Interfaces · Polymorphism · One public class per file · Wrappers · Autoboxing/Unboxing · Integer cache · POJO*